TECNOLOGÍA EN DESARROLLO DE SOFTWARE

PROGRAMACIÓN AVANZADA Y APLICADA

PROYECTO ASISTENTEIA

CONSUMO DE API REST

Contrato, errores y evidencia

Elaborado por	Juan Esteban Olaya Muñoz
Docente	Mg. Sandra Esperanza Sánchez Barragán
Programa	Tecnología en Desarrollo de Software
Materia	Programación Avanzada y Aplicada
Modalidad	Individual
Producto	Contrato, errores y evidencia
Fecha	23/10/2026

1. Proveedor principal

Proveedor principal: Google Gemini mediante su endpoint compatible con Chat Completions (https://generativelanguage.googleapis.com/v1beta/openai/). Documentación: https://ai.google.dev/gemini-api/docs — claves: https://aistudio.google.com/apikey. Proveedor secundario: Pollinations (https://gen.pollinations.ai), con el mismo esquema; solo se verificó sin clave (catálogo público de modelos y error 401).

2. Endpoints

Operación	Método y URL	Autenticación
Catálogo de modelos	GET https://gen.pollinations.ai/v1/models	Pública en Pollinations (la app envía Bearer solo si el proveedor lo exige).
Chat / validación	POST https://gen.pollinations.ai/v1/chat/completions	Authorization: Bearer <API_KEY_REDACTADA>

3. Solicitud (ejemplo sanitizado)

```
POST /v1/chat/completions HTTP/1.1
Host: gen.pollinations.ai
Authorization: Bearer ****REDACTADA****
Content-Type: application/json

{"model":"openai","messages":[{"role":"user","content":"¿Qué es MVVM?"}]}
```

En la prueba de conexión se añade "max_tokens":1 y el contenido "ping".

4. Respuesta (ejemplo ilustrativo, no proviene de una ejecución real)

```
HTTP/1.1 200 OK
{"choices":[{"message":{"role":"assistant","content":"MVVM es un patrón…"}}]}
```

La app solo lee choices[0].message.content; una respuesta sin ese campo se trata como InvalidResponse.

5. Códigos HTTP y manejo

HTTP / situación	AiErrorKind	Mensaje al usuario
200	—	Respuesta mostrada.
400	BadRequest	Solicitud rechazada: revisa el modelo o el contenido.
401 / 403	Unauthorized	Clave inválida o sin permisos; la configuración se marca como no validada.
404	NotFound	Modelo o endpoint no encontrado.
408 / timeout (60 s)	Timeout	El proveedor tardó demasiado.
429	RateLimited	Límite de uso o créditos agotados.
5xx	ServerError	Error temporal del proveedor.
Sin red / DNS	NoConnection	Sin conexión con el proveedor.
JSON inválido o vacío	InvalidResponse	No se pudo interpretar la respuesta.

6. Evidencia real ejecutada (sin API key)

Ejecutada el 23/10/2026 desde este equipo (archivo docs/evidencias/rest_pollinations_sin_clave.txt y pruebas LiveTests):

- GET /v1/models → HTTP 200; 388 modelos totales, 242 de texto; el modelo openai/gpt-5.4-nano figura con paid_only=false y otros con paid_only=true.
- POST /v1/chat/completions sin Authorization → HTTP 401 {"error":{"code":"UNAUTHORIZED",…"A valid API key is required"…}}. Confirma que Pollinations exige clave.
- Prueba en vivo con una clave falsa → AiErrorKind.Unauthorized, HTTP 401 (Live_Pollinations_InvalidKey_Returns401).

Pendiente: una respuesta 200 de chat requiere una API key real, que no estaba disponible; no se simuló ninguna evidencia.

7. Evidencia real con API key (Gemini)

Ejecutada el 23/10/2026 con una clave real de Google AI Studio (archivo docs/evidencias/rest_gemini_con_clave.txt, clave redactada). Base: https://generativelanguage.googleapis.com/v1beta/openai/

Llamada	Resultado real
GET /models	HTTP 200; 60 modelos (los ids llegan como models/<id>; la app quita el prefijo y excluye modelos de imagen, audio, tts y embeddings).
POST /chat/completions, gemini-3.1-flash-lite, max_tokens=1	HTTP 200, finish_reason=length, message sin content (validación correcta).
POST /chat/completions, gemini-3.5-flash, max_tokens=1	HTTP 200, igual: modelo con razonamiento.
POST /chat/completions, gemini-3.1-flash-lite, consulta real	HTTP 200, finish_reason=stop, 29 tokens de salida; respuesta: “MVVM es un patrón de arquitectura de software que separa la lógica de negocio de la interfaz de usuario…”.
Modelo inexistente	HTTP 404 NOT_FOUND.
gemini-2.0-flash (retirado)	HTTP 404: “no longer available”. gemini-2.5-flash y gemini-2.5-flash-lite también dieron 404 para esta cuenta.

8. Seguridad del consumo

- La clave solo se envía en el encabezado Authorization al proveedor elegido o detectado por prefijo local.
- No se registran encabezados, claves ni prompts en logs; los mensajes de error no incluyen la clave (verificado por prueba).
- Sin reintentos automáticos para no duplicar consumo.

9. Referencias consultadas

- Pollinations. Documentación de la API: https://gen.pollinations.ai/
- Google AI for Developers. Gemini API (endpoint compatible con Chat Completions): https://ai.google.dev/gemini-api/docs